

XML850R_AR

Proposed source, changes highlighted | SGSRC/QRPGLESRC | generated 2026-08-26 | NOT applied to the IBM i

■ added or replaced ■ removed (struck through) white = unchanged

Change IDs in this member: C3, C4, C7, C8, C10, C11, C12, C13, C14, C15, C16, C17, C18, C19, C20

Line numbers count the proposed source only; removed lines are unnumbered.

```
1      H Debug(*Yes) TruncNbr(*Yes) AlwNull(*UsrCtl)
2      H DftActGrp(*No) ActGrp(*Caller)
3      H Option(*SrcStmt : *NoDebugIO) ExprOpts(*ResDecPos)
4      H BndDir('SGSRC/XML850R_AR')
5      F*****
6      F* Author          - Chris Hutchinson / Pro I.T. Resource Group, Inc. *
7      F*                  chutchinson@proitrg.com | 800-545-7630 *
8      F*                  www.proitrg.com | 336-420-0329 *
9      F* Program Name   - XML850R_AR *
10     F* Program Date   - 05/14/2019 *
11     F* Source Library - QRPGLESRC/SGSRC *
12     F* Object Library - SGPGM *
13     F* Program Descr  - Process incoming cXML order transactions for *
14     F*                  imported orders - new only - change requests *
15     F*                  have not been implemented yet. *
16     F* *
17     F* NOTE: Elements with attributes were broken down as on page 67 *
18     F*       of Scott Klement's "Web Services and XML for RPGers.pdf" *
19     F*       at 'https://www.scottklement.com/presentations/Web%20 *
20     F*       Services%20and%20XML%20for%20RPGers.pdf' *
21     F* *
22     F*****
23     F* *
24     F* MODIFICATIONS *
25     F* *
26     F* 10/14/2020 CDH Modified the path handling to use the download *
27     F*                  path specified in ARPEGGIO SFTP software for *
28     F*                  the server definition defined for the retailer. *
29     F* *
30     F* 06/05/2023 CDH Modified the date required for the order coming *
31     F*                  in to be handled with the new logic Bill *
32     F*                  provided. It is now based on the part class or *
33     F*                  the item to determine the number of days added *
34     F*                  to the date required. It also accounts for *
35     F*                  holidays as specified in the user defined code *
36     F*                  table. *
37     F* *
38     F*****
39
40     * HarrisData Customer Master File
41     FHDCUST IF E K Disk
42
43     * HarrisData Item Master File
```

```

44      FHDIMST    IF    E            K Disk
45
46      * HarrisData Item Plant Master File
47      FHDIPLT    IF    E            K Disk
48
49      * HarrisData Iterm Warehouse Master File
50      FHDIWHS    IF    E            K Disk
51
52      * HarrisData Order Flags Master
53      FHDORFG    IF    E            K Disk
54
55      * HarrisData Ship Via Master
56      FHDSHPV    IF    E            K Disk
57
58      * HarrisData Cost Summary Master File
59      FHDMCMM    IF    E            K Disk
60
61      * HarrisData Tax Definition File
62      FHD TAXD    IF    E            K Disk
63
64      * HarrisData Order Header Work File
65      FOEHDWK    UF A E            K Disk
66
67      * HarrisData Order Detail Work File
68      FOEDTWK    O    E            Disk    Block(*NO)
69
70      * HarrisData Order Comment Work File
71      FOECMWK    UF A E            K Disk    Block(*NO)
72
73      * Arpeggio SFTP Software - Server Definition File
74      * (for download path)
75      FAFSVR1P    IF    E            K Disk
76
77      * XML Orders Header Master
78      FXMLOEHD    O    E            Disk    Block(*NO)
79
80      * XML Orders Header Master
81      FXMLOEDT    O    E            Disk    Block(*NO)
82
83      * Program Fields *
84      D NumParams      s            1s 0
85      D ProcessMode     s            1a
86      D fileName        s            256a
87      D ifsFile         s            256a
88      D Length          s            3s 0
89      D RestofLen       s            3s 0
90      D orderCounter    s            2s 0
91      D lineCounter     s            3s 0
92      D BuyerOrdLn      s            5s 0
93      D TempID          s            12a
94      D TempDate        s            d    DatFmt(*ISO)    OEHDWK/H1SHTO
95      - C10 D AddDays s 2s 0
96      + C10 D AddDays    s            5s 0
97      D DaysOff         s            7s 0
98      D TodayISO        s            d    DatFmt(*ISO)

```

98	D LastDt1Date	s	d	DatFmt(*ISO)
99	D posstart	s	3s 0	
100	D X	s	5s 0	
101	D F	s	2s 0	
102	D DFL	s	1a	Dim(20)
103	D APID	s	2a	Inz('OE')
104	D FGORTY	s	1a	
105	D Day_of_Week	s	1s 0	
106	D HDPCust#	s	7s 0	
107	D xmlMode	s	1a	
108	D xmlFASetID	s	10a	
109	D xmlFATransID	s	10a	
110	D OrdTotal	s	11p 2	
111	D DQ#NAM	s	10a	Inz('OEDTAQ')
112	D DQ#LIB	s	10a	Inz('*LIBL')
113	D DQ#LEN	s	5s 0	Inz(81)
114	D GiftMSG	s	256a	
115	D ShipToFnd	s	1a	Inz('N')
116	D STTxCd	s	1a	
117	D STPlaceId	s	20a	
118	D STName1	s	30a	
119	D STName2	s	30a	
120	D STNameFL	s	30a	
121	D STAdr1	s	30a	
122	D STAdr2	s	30a	
123	D STAdr3	s	30a	
124	D STCity	s	30a	
125	D STState	s	2a	
126	D STZip	s	10a	
127	D STCountry	s	3a	
128	D STEmail	s	100a	
129	D STPhone	s	15a	
130	D BillToNum	s	7s 0	
131	D BTPlaceId	s	20a	
132	D BTName1	s	30a	
133	D BTName2	s	30a	
134	D BTAdr1	s	30a	
135	D BTAdr2	s	30a	
136	D BTAdr3	s	30a	
137	D BTCity	s	30a	
138	D BTState	s	2a	
139	D BTZip	s	10a	
140	D BTCntry	s	3a	
141	D BTEmail	s	100a	
142	D BTPhone	s	15a	
143	D CSPlaceId	s	20a	
144	D CSName1	s	30a	
145	D CSName2	s	30a	
146	D CSAdr1	s	30a	
147	D CSAdr2	s	30a	
148	D CSAdr3	s	30a	
149	D CSCity	s	30a	
150	D CSState	s	2a	
151	D CSZip	s	10a	
152	D CSEmail	s	100a	

153		D CSPhone	s	15a	
154		D IVPlaceId	s	20a	
155		D IVName1	s	30a	
156		D IVName2	s	30a	
157		D IVAdr1	s	30a	
158		D IVAdr2	s	30a	
159		D IVAdr3	s	30a	
160		D IVCity	s	30a	
161		D IVState	s	2a	
162		D IVZip	s	10a	
163		D IVEmail	s	100a	
164		D IVPhone	s	15a	
165		D UsePath	s	15a	
166		D Cmd	s	500a	
167		D CmdLen	s	15p 5	
168		D DropShip	s	7s 0	
169		D FAPath	s	155a	Varying
170		D FAFile	s	100a	Varying
171		D FAFull	s	256a	Varying
172		D RightNow	s	Z	
173		D Avail	s	13p 4	
174		D from	s	100a	
175		D To	s	100a	
176		D replyTo	s	100a	
177		D subject	s	100a	
178		D text	s	9999a	
179		D distribution	s	100a	
180		D email	s	0	class(*java:RPGMail)
181		D Alert	s	10a	
182		D AlertSubj	s	200a	
183		D AlertMsg	s	9999a	
184		D pass	s	10i 0	
185		D StockItem	s	1a	
186		D orderAction	s	10a	
187		D CMSVS	s	2a	
188		D WorkDate	s	6a	
189		D NullInd	s	2b 0	
190		D UDCDays	s	2s 0	
191	+	C10	D WkClass	s	Like(IMPCLS)
192	+	C10	D CustClass	s	Like(CMCCLS)
193	+	C10	D LeadDays	s	5s 1
194	+	C10	D ReqDays	s	5s 0
195					
196		D XMLorderId	s	20a	
197		D LastOrder	s	8s 0	
198		D CurDate	s	D	DatFmt(*ISO)
199		D CurTime	s	T	
200		D Transact	s	10a	
201		D HWLinCnt	s	5s 0	
202		D lineTotal	s	10i 0	
203		D OrderTotal	s	10i 0	
204		D CustomItem	s	1a	
205		D DebugFlag	s	1a	
206		D SaveDesc	s	300a	
207		D CharPos	s	3s 0	

208	D ProcessedItems	s	3s 0	
209				
210	D 01IMDS	s	30a	
211	D 01BITEM	s	35a	
212	D 01VITEM	s	35a	
213	D 01ITEM	s	15a	
214	D 01CUOM	s	3a	Inz('EA')
215	D 01PUOM	s	3a	Inz('EA')
216	D 01QUOM	s	3a	Inz('EA')
217	D 01ORL#	s	3P 0	
218	D 01QORD	s	13p 4	
219	D 01RQDTA	s	25a	
220	D 01RQDTD	s	D	DatFmt(*ISO-)
221	D 01COST	s	13p 5	
222	D 01SLPR	s	13p 5	
223	D 01SLPRXLT	s	13a	
224	D 01SVSV	s	2a	
225	D 01CNCDT	s	8a	
226	D 01DLVDT	s	8a	
227				
228	D ErrFlag	s	1a	
229	D ErrDescr	s	100a	
230	D RecsWrtn	s	1a	
231				
232				
233	D HWPO#	s	22a	OEHDWK/H1ORRF
234	D HWOrdDate	s	25a	yyyymmdd
235	D HWMerchDpt	s	10a	
236	D HWShpToID	s	15a	OEHDWK/H1SHTO
237	D HWBillToID	s	12a	OEHDWK/H1BLTO
238	D HWLineCnt	s	3s 0	
239	D HWCustID	s	12a	OEHDWK/H1BLTO
240	D HWInvToID	s	12a	OEHDWK/H1BLTO
241	D HWShipCd	s	15a	OEHDWK/H1SVSV-H1SVDS
242	D HWControl	s	5a	
243	D HWSlsDiv	s	10a	
244	D HWCustOrd	s	15a	???????
245	D HWBuyCtrct	s	10a	???????
246	D HWReqShpD	s	10a	
247	D HWPOType	s	2a	
248	D HWGiftMSG	s	256a	
249				
250	D xmlWHnd1	s	64a	Inz(*blanks)
251	D xmlOCTL	s	8p 0	
252	D xmlFRT	s	9p 2	
253	D xmlCHRG	s	9p 2	
254	D xmlUPDTOT	s	1a	
255				
256	D CustOrdLn	s	3s 0	
257	D CustRqDt	s	7s 0	
258	D CustDesc	s	30a	
259	D CustCmt	s	60a	
260	D CustQOrd	s	13s 4	
261	D CustSlPr	s	13s 5	
262				

```

263      * XML Acknowledgement Variables *
264
265      * Customer#
266      D XMLACKCUST      s          20A
267      * Acknowledgement Date (YYYY-MM-DD)
268      D XMLACKDATE      s          D    datfmt(*ISO)
269      * Ack. Order ID
270      D XMLACKORDR      s          20A
271      * Ack. Customer PO#
272      D XMLCUSTPO       s          22A
273      * Ack. Type
274      D XMLACKTYPE      s          6A
275      * Customer PO Date
276      D XMLCUSTPD       s          D    datfmt(*ISO)
277
278
279      D HOEUMC_W         PR          ExtPgm('HOEUMC_W')
280      D xmlWHnd1         64a
281      D xmlDBID          2a
282      D xmlOCTL          8p 0
283      D xmlFRT           9p 2
284      D xmlCHRG          9p 2
285      D xmlUPDTOT        1a
286
287      D Ack              PR          ExtPgm('XMLACKR')
288      D xmlRetailer      15a
289      D HWPO#            22a
290      D ErrFlag          1a
291      D ErrDescr         100a Options(*NoPass)
292
293      D XML850R_AR       PR          ExtPgm('XML850R_AR')
294      D xmlDBID          2a
295      D xmlRetailer      15a
296      D xmlFile          100a
297      D xmlSvrDfn        10a Options(*NOPASS)
298
299      D XML850R_AR       PI
300      D xmlDBID          2a
301      D xmlRetailer      15a
302      D xmlFile          100a
303      D xmlSvrDfn        10a Options(*NOPASS)
304
305      D error            DS          LikeDS(Error_Info)
306
307      D DTAQ             E DS          ExtName(OEDTAQPL)
308
309
310      * Prototype for Send Data Queue (QSNDDTAQ) API
311      D QSNDDTAQ         PR          ExtPgm('QSNDDTAQ')
312      D DQ#NAM           10a Const
313      D DQ#LIB           10a Const
314      D DQ#LEN           5p 0 Const
315      D Data             32766a Options(*varsize)
316
317      * QCmdExc Parameters *

```

```

318 D ExecCmd          PR              EXTPGM('QCMDEXC')
319 D Cmd              500a
320 D CmdLen           15p 5
321
322 * Prototypes for RPGMail *
323 /copy RPGMail/qsource,RPGMailCP
324 /copy RPGMail/qsource,ErrorCP
325
326 // QMHSNDPM prototype + work fields - log alert send failures (3c)
327 dcl-pr SndPgmMsg ExtPgm('QMHSNDPM');
328     p_MsgID      char(7)      const;
329     p_MsgFile     char(20)     const;
330     p_MsgData     char(32767)  const options(*varsize);
331     p_MsgDataLen  int(10)      const;
332     p_MsgType     char(10)     const;
333     p_CallStkEnt  char(10)     const;
334     p_CallStkCnt  int(10)      const;
335     p_MsgKey      char(4);
336     p_ErrCode     char(32767)  options(*varsize);
337 end-pr;
338
339 dcl-s MsgKey char(4);
340 dcl-s LogTxt char(512);
341 dcl-ds ApiErr qualified;
342     BytesProv int(10) inz(8);
343     BytesAvail int(10) inz(0);
344 end-ds;
345
346
347
348
349
350 *****
351 ** XML Data TEMPLATES **
352 *****
353
354 D*-----
355 D*| Header Template |
356 D*-----
357
358 dcl-ds Sender_T Qualified Template ;
359     dcl-ds Credential likeDS(Credential_T) ;
360     UserAgent Char(20) ;
361 // end-ds ;      ---> NOT specified after a DS is declared with likeDS!!!
362 end-ds Sender_T ;
363
364 dcl-ds Credential_T Qualified Template ;
365     domain      Char(20) ;
366     Identity     Char(20) ;
367     SharedSecret Char(15) ;
368 end-ds;
369
370 *****
371 * XML Data Structure Definitions *
372 *****

```

```

373 dcl-ds cXML Qualified ;
374     payloadID      Char(50) ;
375     version        Char(10) ;
376
377 dcl-ds Header ;
378
379     dcl-ds From ;
380         dcl-ds Credential ;
381             domain   Char(20) ;
382             Identity Char(20) ;
383         end-ds Credential ;
384     end-ds From ;
385
386     dcl-ds To ;
387         dcl-ds Credential ;
388             domain   Char(20) ;
389             Identity Char(20) ;
390         end-ds Credential ;
391     end-ds To ;
392
393     dcl-ds Sender ;
394         dcl-ds Credential likeDS(Credential_T) inz(*likeDS) ;
395 //     end-ds Credential ;      ---> Not used on a data structure declared with likeDS!!!
396     end-ds Sender ;
397
398 end-ds Header ;
399
400 dcl-ds Request ;
401     deploymentMode Char(10) ;
402
403     dcl-ds OrderRequest Dim(50);
404     dcl-ds OrderRequestHeader ;
405         orderDate      Char(25) ;
406         orderID         Char(15) ;
407         numoforderID    Int(10);
408         orderType       Char(8) ;
409         orderVersion    Zoned(2:0) ;
410         type            Char(10) ;
411     dcl-ds Total ;
412         dcl-ds Money ;
413             alternateAmount Char(10) ;      // Was Zoned(8:2) but they don't send it
414             alternateCurrency Char(10) ;
415             currency        Char(3) ;
416             dsvalue         Char(50) ;
417         end-ds Money ;
418     end-ds Total ;
419     dcl-ds ShipTo ;
420         dcl-ds Address ;
421             addressID      Char(25) ;
422             isoCountryCode Char(5) ;
423         dcl-ds Name ;
424             xml_lang       Char(2);          // Was xml:lang; Works w/case convert
425             dsvalue        Char(50);
426         end-ds Name ;
427     dcl-ds PostalAddress ;

```

```

428         name      Char(10) ;
429         DeliverTo Char(30) Dim(2) ;
430         Street    Char(30) ;
431         City      Char(15) ;
432         State     Char(2) ;
433         PostalCode Char(10) ;
434         dcl-ds Country ;
435             isoCountryCode Char(2) ;
436             dsvalue      Char(50) ;
437         end-ds Country ;
438     end-ds PostalAddress ;
439     dcl-ds Email ;
440         name      Char(10) ;
441         preferredLang Char(5) ;
442         dsvalue    Char(50) ;
443     end-ds Email ;
444     dcl-ds Phone ;
445         name      Char(10);
446         dcl-ds TelephoneNumber ;
447             dcl-ds CountryCode ;
448                 isoCountryCode Char(2) ;
449                 dsvalue      Char(1) ;
450             end-ds CountryCode ;
451             AreaOrCityCode Zoned(3:0) ;
452             Number        Zoned(10:0) ;
453         end-ds TelephoneNumber ;
454     end-ds Phone ;
455     dcl-ds Fax ;
456         name      Char(10) ;
457         dcl-ds TelephoneNumber ;
458             dcl-ds CountryCode ;
459                 isoCountryCode Char(2) ;
460                 dsvalue      Char(1) ;
461             end-ds CountryCode ;
462             AreaOrCityCode Zoned(3:0) ;
463             Number        Zoned(10:0) ;
464         end-ds TelephoneNumber ;
465     end-ds Fax ;
466     end-ds Address;
467 end-ds ShipTo;
468
469 dcl-ds BillTo ;
470     dcl-ds Address ;
471         addressID      Char(4) ;
472         isoCountryCode Char(2) ;
473     dcl-ds Name ;
474         xml_lang      Char(2) ;
475         dsvalue      Char(32) ;
476     end-ds Name ;
477     dcl-ds PostalAddress ;
478         name      Char(30) ;
479         Street    Char(50) ;
480         City      Char(30) ;
481         State     Char(2) ;
482         PostalCode Char(10) ;

```

```

483         dcl-ds Country ;
484             isoCountryCode Char(2) ;
485             dsvalue Char(25) ;
486         end-ds Country ;
487     end-ds PostalAddress ;
488     end-ds Address ;
489 end-ds BillTo ;
490 dcl-ds PaymentTerm ;
491     payInNumberOfDays Zoned(3:0) ;
492 end-ds PaymentTerm ;
493 Comments Char(250) ;
494 dcl-ds Extrinsic ;
495     name Char(15) ;
496     dsvalue Char(50) ;
497 end-ds Extrinsic ;
498 end-ds OrderRequestHeader ;
499
500 dcl-ds ItemOut Dim(500) ;
501     quantity Char(13) ;
502     requestedDeliveryDate Char(25) ;
503     lineNumber Zoned(5:0) ;
504 dcl-ds ItemID ;
505     SupplierPartID Char(15) ;
506     SupplierPartAuxiliaryID Char(15) ;
507 end-ds ItemID ;
508
509 dcl-ds ItemDetail ;
510     dcl-ds UnitPrice ;
511         dcl-ds Money ;
512             alternateCurrency Char(5) ;
513             alternateAmount Char(10) ; // Was Zoned(5:2) but they don't provide it
514             currency Char(3) ;
515             dsvalue Char(13) ;
516         end-ds Money ;
517     end-ds UnitPrice ;
518     dcl-ds Description ;
519         xml_lang Char(2) ;
520         dsvalue Char(300) ;
521     end-ds Description ;
522     UnitOfMeasure Char(2) ;
523 dcl-ds Classification ;
524     domain Char(10) ;
525     Classification Char(10) ;
526 end-ds Classification ;
527 ManufacturerPartID Char(15) ;
528 dcl-ds Extrinsic ;
529     name Char(25) Dim(5) ;
530 end-ds Extrinsic ;
531
532 end-ds ItemDetail ;
533 end-ds ItemOut ;
534
535
536 end-ds OrderRequest ;
537 end-ds Request ;

```

```

538         end-ds cXML ;
539
540
541
542     * Constants *
543     D Up          C          'ABCDEFGHIJKLMNOPQRSTUVWXYZ'
544     D Lo          C          'abcdefghijklmnopqrstuvwxyz'
545     D CrLf        C          Const(x'0d25')
546
547     * Search/Replace Constants
548     - C19 D apos      C          'andapos;'
549     + C19      D apos      C          '&apos; '
550     - C19 D quot       C          'andquot;'
551     + C19      D quot      C          '&quot; '
552     - C19 D amper      C          'andamp;'
553     + C19      D amper     C          '&amp; '
554     - C19 D lt         C          'andlt;'
555     + C19      D lt        C          '&lt; '
556     - C19 D gt         C          'andgt;'
557     + C19      D gt        C          '&gt; '
558
559
560     * ----- Main Module -----*
561
562     /Free
563
564     // Prepare email for alerts...
565     Monitor;
566     RPGMail_init();
567     On-error;
568     error = Error_catch();
569     EndMon;
570
571     // Reset the error flag.
572     ErrFlag = 'N';
573
574     // Check the number of parameters. 4 parameters mean this is a
575     // production run. The 4th parameter is the server definition
576     // used by ARPEGGIOL. This is not passed if this is a test run
577     // of an archived inbound order.
578     NumParams = %Parms;
579     If NumParams < 4;
580         // Testing an /orders/archived inbound/amerigas/orders/
581         // {filename} order
582         ProcessMode = 'T';
583     Else;
584         // Executing a production run of an amerigas order
585         ProcessMode = 'L';
586     EndIf;
587
588     Exsr getXMLData;
589
590     // Process UP TO 50 orders per cXML file (per the "Dim"
591     // statement on the SGOrder Data Structure spec)

```

```

588      Exsr loadDB2Data;
589
590      // If this is a production order, archive the order.
591      If ProcessMode = 'L';
592          Exsr exitProgram;
593      EndIf;
594
595      *InLR = *On;
596
597
598      //-------------------------------------*
599      // getXMLData Subroutine will pull all XML variables out and    *
600      // put them into their appropriate production data fields        *
601      //-------------------------------------*
602
603      BegSr getXMLData;
604
605          /** * * * * * * * * * * */
606          /** This runs for AMERIGAS only **
607          /** * * * * * * * * * * */
608          BillToNum = 1590000;
609          H1B1TO   = 1590000;
610
611          usepath = XMLRetailer;
612
613          // Pull extraneous data required based on the trading
614          // partner being processed...
615          Exec SQL
616              Select Substr(UDCDESC3,1,1), Dec(UDCDESC2,7,0)
617                  Into :xmlMode, :HDP Cust#
618                  From ProITRG/UDCDETAIL
619                  Where UDCSYSTEMD = 'XML' And UDCKEY = :XMLRetailer;
620
621          If SQLCod <> 0;
622              // Notify somebody of an error trying to process :XMLRetailer!!!
623              // Don't know how this could happen if the user got this far.
624              Alert = 'XMLNOCUST';
625              Select;
626              When ProcessMode = 'T';
627                  AlertSubj = '*** Testing Archived Inbound Order file ' +
628                              %Trim(xmlFile) + ' ***';
629              When xmlMode = 'T';
630                  AlertSubj = '*** Test Order file ' + %Trim(xmlFile) +
631                              ' sent in by ' + %Trim(xmlRetailer) +
632                              '***';
633              Other;
634                  AlertSubj = 'Republic Order ' + %Trim(xmlFile) +
635                              ' Alert';
636          EndSL;
637
638          AlertMsg = %Trim(AlertMsg) +
639                      '--> This file cannot be processed. ' +
640                      'No HD primary customer number in the User' +
641                      ' Defined Codes file for ' +
642                      %Trim(xmlretailer) + '.';

```

```

643         Exsr Notify ;
644         *InLR = *On;
645         Return;
646     EndIf;
647
648     // Pull the path used for the download to extract the filename
649     // if this is a production order.
650     If ProcessMode = 'L';
651         Chain xmlSvrDfn AFSVR1P ;
652
653         Length = %len(%Trim(SFDWNFLR)) ;
654         If Length = 0 ;
655             - C11 Alert = 'XMLORDERS';
656             + C11 Alert = 'XMLORDERR';
657             AlertSubj = 'Amerigas order file ' + %Trim(xmlFile) +
658                 ' Alert';
659
660             AlertMsg = 'This file cannot be processed. ' +
661                 'No path in the ' + %Trim(xmlSvrDfn) +
662                 ' server definition in ARPSFTP.' ;
663             Exsr Notify ;
664             *InLR = *On ;
665             Return ;
666         EndIf;
667     EndIf;
668
669     // ProcessMode takes precedence over xmlMode because
670     // ProcessMode is what we're doing at a higher level
671     // ...production order a testing an order from archived
672     // inbound. xmlMode is which we're processing for the
673     // retailer - a test order they sent or a production order
674     // they sent.
675     Select;
676     When ProcessMode = 'T' Or xmlMode = 'T';
677         filename = '/orders/archive inbound/' +
678             %Trim(xmlRetailer) + '/orders/' +
679             %Trim(xmlFile);
680     Other;
681         RestofLen = %len(xmlFile) - Length;
682         ifsFile = %Subst(xmlFile:Length + 1:RestofLen);
683         filename = '/orders/inbound/' +
684             %Trim(xmlRetailer) + '/' + %Trim(ifsFile);
685
686     EndSL;
687
688
689     //-----*
690     // Original Code. XML-INTO can't handle the "NIL" variables so this *
691     // was changed to a DATA-INTO opcode with the XMLNSC specified which *
692     // CAN handle the "NIL" element values. (ex. <AreaOrCityCode/>) *
693     //-----*
694     // *
695     // XML-INTO(EH) cXML %xml(%Trim(fileName): *
696     // 'case=convert doc=file datasubf=dsvalue + *
```

```

697 //          allowextra=yes allowmissing=yes + *
698 //          countprefix=numof ns=merge'); *
699 // *
700 //-----*
701
702 // DATA-INTO cXML %Data(fileName : *
703
704 XML-INTO cXML %xml(%Trim(fileName):
705 'case=convert doc=file datasubf=dsvalue +
706 allowextra=yes allowmissing=yes +
707 countprefix=numof ns=merge');
708
709 EndSr;
710
711 //-----*
712 // loadDB2Data subroutine will populate all the data required *
713 // for HarrisDatas' normal order processing. *
714 //-----*
715
716 BegSr loadDB2Data;
717
718 orderTotal = cXML.Request.OrderRequest(1).
719 OrderRequestHeader.numofOrderID;
720
721 For orderCounter = 1 to orderTotal;
722
723 // Check to see if this is a Change or Cancel request.
724 // If it is, just email customer service and don't
725 // process the order.
726 If %Trim(cXML.Request.OrderRequest(orderCounter).
727 OrderRequestHeader.orderType) = 'regular' AND
728 %Trim(cXML.Request.OrderRequest(orderCounter).
729 OrderRequestHeader.type) = 'new';
730
731 Exsr writeOrder;
732
733 Else;
734
735 // Notify Customer Service of Order Type we
736 // can't handle...
737
738 orderAction = cXML.Request.OrderRequest(orderCounter).
739 OrderRequestHeader.type;
740
741 // Convert orderAction to uppercase with SQL
742 Exec SQL
743 Set :orderAction = Upper(:orderAction);
744
745 Alert = 'ORDUPDATE';
746 AlertSubj = 'Americas Order Alert for file ' +
747 %Trim(xmlFile) + '. Order type ' +
748 %Trim(orderAction) +
749 ' - PO# ' + %Trim(%Char(cXML.Request.
750 OrderRequest(orderCounter).
751 OrderRequestHeader.orderID));

```

```

752         AlertMsg = 'Program XML850R_AR does not know it ' +
753                   'should process this order type.';
754         Exsr Notify;
755
756     EndIf;
757
758 EndFor;
759
760 EndSr;
761 //-----*
762
763
764
765 //-----*
766 /* writeOrder loads the fields for the transaction header
767 //-----*
768 BegSr writeOrder;
769
770     Clear OEHDWKR;
771     Clear XMLOEHDR;
772
773     // Write the header for the order...
774     XMLOrderID = cXML.Request.OrderRequest(orderCounter).
775                 OrderRequestHeader.orderID;
776
777     Exsr NxtHDOOrd;
778
779     HWPO# = cXML.Request.
780             OrderRequest(orderCounter).
781             OrderRequestHeader.orderID;
782     HWPO# = %XLate(Lo:Up:HWPO#);
783
784 - C18 ----- If HWPO# = '410022893';
785 - C18 ----- DebugFlag = 'Y';
786 - C18 ----- EndIf;
787 + C18 // C18 - leftover debug, DebugFlag is never read
788 + C18 // If HWPO# = '410022893';
789 + C18 // DebugFlag = 'Y';
790 + C18 // EndIf;
791
792     HWOOrdDate = %Char(%Date(%Subst(cXML.Request.
793                                     OrderRequest(orderCounter).
794                                     OrderRequestHeader.orderDate:1:10):*ISO-));
795     If %Upper(%Subst(cXML.Request.OrderRequest(orderCounter).
796                     OrderRequestHeader.ShipTo.Address.addressID:1:7)) =
797         'ADBUYER';
798         HWShpToID = %Subst(cXML.Request.OrderRequest(orderCounter).
799                             OrderRequestHeader.ShipTo.Address.addressID:8:15);
800     Else;
801         HWShpToID = cXML.Request.
802                     OrderRequest(orderCounter).
803                     OrderRequestHeader.ShipTo.
804                     Address.addressID;
805     EndIf;
806     If cXML.Request.

```

```

804         OrderRequest(orderCounter).
805         OrderRequestHeader.ShipTo.
806         Address.PostalAddress.
807         DeliverTo(1) <> *Blanks;
808         STName1 = %Trim(cXML.Request.
809             OrderRequest(OrderCounter).
810             OrderRequestHeader.ShipTo.Address.
811             PostalAddress.DeliverTo(1));
812     Else;
813         STName1 = %Trim(cXML.Request.
814             OrderRequest(OrderCounter).
815             OrderRequestHeader.ShipTo.Address.
816             Name.dsvalue);
817     EndIf;
818
819     STName1 = %XLate(Lo:Up:STName1);
820     STAdr1 = cXML.Request.
821         OrderRequest(orderCounter).
822         OrderRequestHeader.ShipTo.
823         Address.PostalAddress.DeliverTo(2);
824     STAdr1 = %XLate(Lo:Up:STAdr1);
825     STAdr2 = cXML.Request.
826         OrderRequest(orderCounter).
827         OrderRequestHeader.ShipTo.
828         Address.PostalAddress.Street;
829     STAdr2 = %XLate(Lo:Up:STAdr2);
830     // STAdr3 = cXML.Request.
831     //     OrderRequest(orderCounter).
832     //     OrderRequestHeader.ShipTo.
833     //     Address.PostalAddress.Street;
834     // STAdr3 = %XLate(Lo:Up:STAdr3);
835     STAdr3 = *Blanks;
836     STCity = cXML.Request.
837         OrderRequest(orderCounter).
838         OrderRequestHeader.ShipTo.
839         Address.PostalAddress.City;
840     STCity = %XLate(Lo:Up:STCity);
841     STState = cXML.Request.
842         OrderRequest(orderCounter).
843         OrderRequestHeader.ShipTo.
844         Address.PostalAddress.State;
845     STState = %XLate(Lo:Up:STState);
846     STZip = cXML.Request.
847         OrderRequest(orderCounter).
848         OrderRequestHeader.ShipTo.
849         Address.PostalAddress.PostalCode;
850     STCountry = cXML.Request.
851         OrderRequest(orderCounter).
852         OrderRequestHeader.ShipTo.
853         Address.PostalAddress.
854         Country.isoCountryCode;
855     STCountry = %XLate(Lo:Up:STCountry);
856
857     Monitor;
858     // All ShipTos won't provide a phone number

```

```

859         STPhone    = %Char(cXML.Request.
860             OrderRequest(orderCounter).
861             OrderRequestHeader.ShipTo.
862             Address.Phone.TelephoneNumber.
863             AreaOrCityCode) +
864             %Char(cXML.Request.
865             OrderRequest(orderCounter).
866             OrderRequestHeader.ShipTo.
867             Address.Phone.TelephoneNumber.
868             Number);
869
870     On-Error;
871     STPhone = *Blanks;
872     EndMon;
873
874     BTName1    = cXML.Request.
875             OrderRequest(orderCounter).
876             OrderRequestHeader.BillTo.
877             Address.Name.dsvalue;
878     BTName1    = %XLate(Lo:Up:BTName1);
879     BTAdr1     = cXML.Request.
880             OrderRequest(orderCounter).
881             OrderRequestHeader.BillTo.
882             Address.PostalAddress.Street;
883     BTAdr1     = %XLate(Lo:Up:BTAdr1);
884     BTCity     = cXML.Request.
885             OrderRequest(orderCounter).
886             OrderRequestHeader.BillTo.
887             Address.PostalAddress.City;
888     BTCity     = %XLate(Lo:Up:BTCity);
889     BTState    = cXML.Request.
890             OrderRequest(orderCounter).
891             OrderRequestHeader.BillTo.
892             Address.PostalAddress.State;
893     BTState    = %XLate(Lo:Up:BTState);
894     BTZip      = cXML.Request.
895             OrderRequest(orderCounter).
896             OrderRequestHeader.BillTo.
897             Address.PostalAddress.PostalCode;
898     BTCntry    = cXML.Request.
899             OrderRequest(orderCounter).
900             OrderRequestHeader.BillTo.
901             Address.PostalAddress.
902             Country.isoCountryCode;
903     BTCntry    = %XLate(Lo:Up:STCountry);
904
905     // Determine the ShipTo as follows...
906     // (Preferred)
907     // - Pull the Customer whose addressID in the cXML document
908     //   that matches the CMUDF4 field on HDCUST and has a BillTo
909     //   of :BillToNum
910     // (Secondary option)
911     // - If that doesn't exist, pull the Customer whose addressID
912     //   in the cXML document matches the CMUDF4 field on HDCUST
913     //   and use that BillTo and ShipTo

```

```

914 // (Tertiary option)
915 // - If neither of those exist, set up the order as a drop
916 // ship with the information provided in the cXML document
917 // and notify orders@screen-graphics.com of the issue.
918
919 ShipToFnd = 'N';
920
921 Exec SQL
922 Select CMCUST, CMSV, CMCTXC Into :H1ShTo, :CMSVSV, :STTXCD
923 From HDCUST
924 Where CMUDF4 = :HWShpToID
925 And CMBLTO = :BillToNum;
926
927 If SQLCod = 0;
928 ShipToFnd = 'Y';
929 EndIf;
930
931 If ShipToFnd = 'N';
932 Exec SQL
933 Select CMCUST, CMBLTO, CMSV, CMCTXC
934 Into :H1ShTo, :H1B1To, :CMSVSV, :STTXCD
935 From HDCUST
936 Where CMUDF4 = :HWShpToID;
937
938 If SQLCod = 0;
939 ShipToFnd = 'Y';
940 EndIf;
941 EndIf;
942
943 If ShipToFnd = 'N';
944 H1ShTo = BillToNum;
945
946 // Figure the Drop Ship Number for this Order
947 Exec SQL
948 Select DSNMBR Into :DropShip
949 From HDDSHP
950 Where DSVCF = 'C' And DSVNCS = :HDPCust#
951 And DSNAME = :STName1 And DSADR1 = :STADR1
952 And DSADR2 = :STADR2 And DSADR3 = :STADR3
953 And DSCITY = :STCITY And DSST = :STSTATE
954 And DSZIP = :STZIP And DSCTRY = :STCOUNTRY
955 And DSCNTC = :CMCNTC And DSCTTC = :CMCTTC
956 And DSLOC1 = :CMLOC1 And DSLOC2 = :CMLOC2
957 And DSLOC3 = :CMLOC3;
958
959 If SQLCod <> 0;
960
961 Exec SQL
962 Select Max(DSNMBR) Into :DropShip from HDDSHP;
963
964 DropShip += 1;
965
966 Exec SQL
967 Insert Into HDDSHP
968 (DSVCF, DSVNCS, DSNMBR, DSNAME, DSADR1, DSADR2, DSADR3,

```

```

969         DSCITY, DSST, DSZIP, DSCTRY, DSCNTC, DSCTTC, DSLOC1,
970         DSLOC2, DSLOC3, DSDATE, DSPURG, DSNAMEU, DSADR1U,
971         DSADR2U, DSADR3U, DSCITYU)
972     Values('C', :HDPCust#, :DropShip, :STName1, :STAdr1,
973           :STAdr2, :STAdr3, :STCity, :STState, :STZip,
974           :STCOUNTRY, :CMCNTC, :CMCTTC, :CMLOC1, :CMLOC2,
975           :CMLOC3, :H1RQDT, 'Y', UCASE(:STName1),
976           UCASE(:STADR1), UCASE(:STAdr2), UCASE(:STAdr3),
977           UCASE(:STCity));
978
979     H1DSHP = DropShip;
980
981     // *** Notify bbusch@screen-graphics.com of ***
982     // *** the new customer that needs to get ***
983     // *** set up using the drop ship info ***
984     Alert = 'XMLDrpShp';
985     Select;
986     When ProcessMode = 'T';
987         AlertSubj = 'Test file ' + %Trim(xmlFile) +
988                   ' sent by Amerigas added a Drop Ship';
989     Other;
990         AlertSubj = 'Amerigas order added a Drop Ship';
991     EndSL;
992
993     AlertMsg = 'Amerigas file ' + %Trim(xmlFile)
994               + CrLf;
995     AlertMsg = %Trim(AlertMsg) + 'HarrisData Order: '
996               + %Char(LastOrder)
997               + CrLf;
998     AlertMsg = %Trim(AlertMsg) + 'Imported Order: '
999               + HWPO# + CrLf;
1000    AlertMsg = %Trim(AlertMsg) + 'New Drop Ship: '
1001              + %Char(DropShip) + CrLf + CrLf;
1002    AlertMsg = %Trim(AlertMsg) + 'Code: '
1003              + HWShpToID + CrLf;
1004    AlertMsg = %Trim(AlertMsg) + 'Address: '
1005              + %Trim(STName1) + CrLf;
1006    AlertMsg = %Trim(AlertMsg) + '          ' + %Trim(STAdr1) +
1007              CrLf;
1008    AlertMsg = %Trim(AlertMsg) + '          ' + %Trim(STAdr2) +
1009              CrLf;
1010    AlertMsg = %Trim(AlertMsg) + '          ' + %Trim(STAdr3) +
1011              CrLf;
1012    AlertMsg = %Trim(AlertMsg) + '          ' + %Trim(STCity) +
1013              ', ' + %Trim(STState) + ' ' + %Trim(STZip) +
1014              CrLf + CrLf;
1015    AlertMsg = %Trim(AlertMsg) + 'Issue: CMUDF4 info right?'
1016              + CrLf;
1017    Exsr Notify;
1018    Else;
1019
1020        H1DSHP = DropShip;
1021
1022    EndIf;
1023

```

```

1024         EndIf;
1025
1026
1027
1028         CustomItem = 'N';
1029
1030         // Write the detail for the order...
1031 +   C13         // C13 - count the detail lines actually written
1032 +   C13         ProcessedItems = 0;
1033 +   C13
1034         For lineCounter = 1 to 500;
1035
1036             Clear OEDTWKR;
1037             Clear XMLOEDTR;
1038
1039             // If the line is not there it will throw an error, exit
1040             // at that point because all items have been processed.
1041
1042             Monitor;
1043
1044             BuyerOrdLn = cXML.Request.
1045                 OrderRequest(orderCounter).
1046                 ItemOut(lineCounter).
1047                 lineNumber;
1048
1049             // If BuyerOrdLn > 999;
1050             O1OrL# = lineCounter;
1051             // Else;
1052             // O1OrL# = BuyerOrdLn;
1053             // EndIf;
1054
1055             On-Error;
1056             Leave;
1057             EndMon;
1058
1059             O1VItem = cXML.Request.
1060                 OrderRequest(orderCounter).
1061                 ItemOut(lineCounter).ItemID.
1062                 SupplierPartID;
1063             O1VItem = %XLate(Lo:Up:O1VItem);
1064
1065             // Exit the loop if the last line was just processed...
1066             If O1VItem = *Blanks;
1067                 Leave;
1068             EndIf;
1069
1070             O1ITEM = O1VITEM;
1071
1072             O1BItem = cXML.Request.
1073                 OrderRequest(orderCounter).
1074                 ItemOut(lineCounter).ItemID.
1075                 SupplierPartID;
1076             O1BItem = %XLate(Lo:Up:O1BItem);
1077
1078             Monitor;

```

```

1079         01QOrd = %Dec(%XLATE('$*',' ':cXML.Request.
1080             OrderRequest(orderCounter).
1081             ItemOut(lineCounter).
1082             quantity) : 13 : 4);
1083     // On-Error 105;
1084     On-Error;
1085     01QOrd = 1;
1086     EndMon;
1087
1088     01QUOM = 'EA';
1089
1090     // ***** //
1091     // If the item is NOT found on HDIMST it is to be processed as a //
1092     // custom, non-stock item ...per Bill Busch 10/15/2019 //
1093     // ***** //
1094
1095     // Description is longer than HD's OEORDT ODIMDS field, so
1096     // check to see if the description includes IMIMDS and, if
1097     // so write the remainder of the SAVEDESC after the ";" to
1098     // OECMWK.
1099     Chain 01ITEM HDIMST;
1100
1101     // Item is not on HDIMST so set it as a 'CUSTOM ITEM'
1102     If NOT %Found(HDIMST);
1103         CustomItem = 'Y';
1104     + C7 // C7 - a failed chain leaves the prior line's
1105     + C7 // IMPCLS in the record. Never read IMPCLS
1106     + C7 // directly again; use WkClass.
1107     + C7 WkClass = *Blanks;
1108         01OREC = 'N';
1109         01ITEM = 'CUSTOM ITEM';
1110         01PCLS = '9MIS';
1111         01STUM = 'EA';
1112         01PRUM = 'EA';
1113         01ORUF = 1.0000;
1114         01TAXC = 'Y';
1115     Else;
1116         CustomItem = 'N';
1117     + C7 WkClass = IMPCLS; // C7
1118         01IMDS = IMIMDS;
1119         01OREC = 'S';
1120         01PCLS = IMPCLS;
1121         01STUM = IMUOMS;
1122     EndIf;
1123
1124     SaveDesc = cXML.Request.OrderRequest(orderCounter).
1125         ItemOut(lineCounter).ItemDetail.Description;
1126
1127     SaveDesc = %Subst(SaveDesc:3:298);
1128
1129     // Search/Replace any invalid XML characters in the description
1130
1131     // apostrophe...
1132     CharPos = %Scan(apos: SaveDesc);
1133     DoW CharPos > 0;

```

```

1134         SaveDesc = %Replace('': SaveDesc: CharPos:
1135             %len(apos));
1136         CharPos = %Scan(apos: SaveDesc: CharPos + 1);
1137     EndDo;
1138
1139     // quote...
1140     CharPos = %Scan(quot: SaveDesc);
1141     DoW CharPos > 0;
1142         SaveDesc = %Replace('': SaveDesc: CharPos:
1143             %len(quot));
1144         CharPos = %Scan(quot: SaveDesc: CharPos + 1);
1145     EndDo;
1146
1147     // ampersand...
1148     CharPos = %Scan(amper: SaveDesc);
1149     DoW CharPos > 0;
1150         SaveDesc = %Replace('&': SaveDesc: CharPos:
1151             %len(amper));
1152         CharPos = %Scan(amper: SaveDesc: CharPos + 1);
1153     EndDo;
1154
1155     // less than...
1156     CharPos = %Scan(lt: SaveDesc);
1157     DoW CharPos > 0;
1158         SaveDesc = %Replace('<': SaveDesc: CharPos:
1159             %len(lt));
1160         CharPos = %Scan(lt: SaveDesc: CharPos + 1);
1161     EndDo;
1162
1163     // greater than...
1164     CharPos = %Scan(gt: SaveDesc);
1165     DoW CharPos > 0;
1166         SaveDesc = %Replace('>': SaveDesc: CharPos:
1167             %len(gt));
1168         CharPos = %Scan(gt: SaveDesc: CharPos + 1);
1169     EndDo;
1170
1171
1172     Clear OECSWKR;
1173
1174     If CustomItem = 'N';
1175         CharPos = %Scan('; ': SaveDesc);
1176         If CharPos > 0;
1177             SaveDesc = %Subst(SaveDesc : CharPos + 1 :
1178                 %Len(%Trim(SaveDesc)) - CharPos + 1);
1179             Exsr WriteCmt;
1180         EndIf;
1181     Else;
1182         Exsr WriteCmt;
1183     EndIf;
1184
1185     OISIPrXlt = %XLATE('$*', '': cXML.Request.
1186         OrderRequest(orderCounter).
1187         ItemOut(lineCounter).ItemDetail.
1188         UnitPrice.Money.dsvalue);

```

1189

~~- C17 01SlPr = %Dec(01SlPrXlt:13:5);~~

1190 + C17 // C17 - a malformed price used to halt the program.

1191 + C17 // The quantity conversion above is monitored;

1192 + C17 // this one now matches it.

1193 + C17 Monitor;

1194 + C17 01SlPr = %Dec(01SlPrXlt:13:5);

1195 + C17 On-Error;

1196 + C17 01SlPr = 0;

1197 + C17 EndMon;

1198

1199 01CUOM = 'EA';

1200

1201

1202 // Populate the OEDTWK fields...

1203 01OCTL = LastOrder;

1204 01ORL# = lineCounter;

1205

1206 01MNCD = 'A';

1207 01BLKT = 'N';

1208 01PLT = 5;

1209 01TAXC = 'N';

1210 01LPOV = 'Y';

1211 01CPOV = 'Y';

1212 01STKL = 'Y';

1213 01LLTC = 'N';

1214 If H1DSHP = 0;

1215 01DSHP = 'N';

1216 Else;

1217 01DSHP = 'Y';

1218 EndIf;

1219 01GNPO = 'N';

1220

1221 // Get the default warehouse for this trading partner...

1222 Chain HDCust# HDCUSTR;

1223 + C15

1224 + C15 // C15 - a failed chain must not leak the prior

1225 + C15 // customer's ship via, warehouse or location.

1226 + C15 If NOT %Found(HDCUST);

1227 + C15 Clear HDCUSTR;

1228 + C15 EndIf;

1229 + C15

1230 + C15 // C9 - customer class drives the prefix lead-day

1231 + C15 // lookup. No extra I/O, the chain is already here.

1232 + C15 CustClass = CMCCLS;

1233

1234 If ShipToFnd = 'Y';

1235 01SVSV = CMSVSV;

1236 Else;

1237 01SVSV = CMSV;

1238 EndIf;

1239

1240 // Populate 01SVDS on OEDTWK for HOEPIC pick ticket Ship Via

1241 Chain 01SVSV HDSHPV;

1242 If %Found(HDSHPV);

```

1243         01SVDS = SVSVDS;
1244     Else;
1245         01SVDS = *Blanks;
1246     EndIf;
1247
1248     01WH  = CMWH#;
1249     01BOAL = *Blanks;
1250     01OCTL = LastOrder;
1251     01LOC# = CMLOC#;
1252     01SHTO = H1SHTO;
1253     01ORST = '0';
1254
1255
1256     Chain (01ITEM:01WH) HDIWHS;
1257     If %Found(HDIWHS);
1258         01PGRP = IWPGRP;
1259         01PRUM = IWUOMP;
1260         01ORUF = IWPCPP;
1261         01TAXC = IWTXCD;
1262         // Per Bill Busch 10/15/2019 ... ALWAYS USE PLANT 1 for anything
1263         // coming in via XML
1264         // 01PLT = IWPLT;
1265         01PLT = 1;
1266     Else;
1267         // C8 - a failed chain must not leak the prior
1268         //     line's quantities into Avail.
1269         IWOHQT = 0;
1270         IWQOO = 0;
1271         IWRESQ = 0;
1272         01PGRP = *Blanks;
1273         01PRUM = 'EA';
1274         01PLT = 1;
1275         01ORUF = 1.0000;
1276         01TAXC = STTxCd;
1277     EndIf;
1278
- C3 ----- // Does this item follow the Stock Item naming nomenclature...
- C3 ----- CharPos = %Scan('-', 01BItem : 2);
- C3 -----
- C3 ----- If %Subst(01BItem:1:1) = '9' And CharPos > 0;
- C3 ----- StockItem = 'Y';
- C3 ----- EndIf;
1279 + C3 // C3 - removed with the rest of the stock-name
1280 + C3 //     testing. StockItem was never read.
1281 + C3 // // Does this item follow the Stock Item naming nomenclature...
1282 + C3 // CharPos = %Scan('-', 01BItem : 2);
1283 + C3 //
1284 + C3 // If %Subst(01BItem:1:1) = '9' And CharPos > 0;
1285 + C3 // StockItem = 'Y';
1286 + C3 // EndIf;
1287
1288     Chain (01PLT:01ITEM) HDIPLT;
1289
1290     // Determine Availability for THIS item...
1291     If %Found(HDIPLT);

```

```

1292      Avail = (IWOHQQT + IWQ00) - (IWRESQ + IPCMTO);
1293 +   C4      LeadDays = IPTLT;                      // C4
1294      Else;
1295 +   C4      LeadDays = 0;                          // C4
1296      // If it's not on HDIPLT make Availability = 0
1297      // Item wasn't found on HDIPLT...
1298      Select;
1299      When ProcessMode = 'T';
1300          AlertSubj = 'Testing Amerigas archive order file ' +
1301                  %Trim(xmlFile);
1302      When xmlMode = 'T';
1303          AlertSubj = 'Amerigas sent in test order ' + HWPO#;
1304      Other;
1305          AlertSubj = 'Amerigas Order Alert: Stock Item Not Set Up';
1306      EndSL;
1307      AlertMsg = 'HarrisData Order: '
1308                + %Char(LastOrder) + CrLf;
1309      AlertMsg = %Trim(AlertMsg) + 'Coupa Order: '
1310                + HWPO# + CrLf;
1311      AlertMsg = %Trim(AlertMsg) + 'Order Line: '
1312                + %Char(O1OrL#) + CrLf;
1313      AlertMsg = %Trim(AlertMsg) + 'Item: '
1314                + O1BItem + CrLf;
1315      AlertMsg = %Trim(AlertMsg) + 'Issue: Configuration'
1316                + CrLf;
1317      AlertMsg = %Trim(AlertMsg) +
1318                'Details: Stock item was not found ' +
1319                'on the HarrisData Item Plant file.';
1320
1321 +   C12      // C12 - this send had no Alert code set, so it
1322 +   C12      //      ran on blanks or a stale code and reached
1323 +   C12      //      the wrong list, or nobody.
1324 +   C12      Alert = 'XMLConfig';
1325      Exsr Notify;
1326
1327      Avail = 0;
1328
1329      EndIf;

```

```

-   C4 -
-   C4 ----- // Regardless of whether or not the item follows the naming convention of a stock item,
-   C4 ----- // if this item has available quantity greater than or equal to the ordered quantity
-   C4 ----- // this item will be considered a stock item.
-   C4 -
-   C4 ----- If Avail >= O1Q0Ord;
-   C4 ----- StockItem = 'Y';
-   C4 ----- EndIf;
-   C4 -
-   C4 ----- // (Standalone stock availability alert removed; the short stock
-   C4 ----- // alert now fires once inside the day count rules below.)
-   C4 -
-   C4 -
-   C4 ----- //
-   C4 ----- // Change per Bill on 5/17/2023... add days to item required
-   C4 ----- // date based on availability and custom or non custom item
-   C4 ----- // ... AddDays will control how many days to add to the

```

```

- C4 ----- // current date to determine the calculated required date.
- C4 ----- //
- C4 ----- // Added the use of the UDC table 10/19/2025 because there
- C4 ----- // are some items that aren't following the generic rules.
- C4 ----- // Those items will get the number of days specified in the
- C4 ----- // UDC table added where everything else will fall through
- C4 ----- // the generic logic defined here.
- C4 ----- //
- C4 -
- C4 -
- C4 ----- // Clear AddDays for each item so CalcReqD is accurate for each line
- C4 ----- AddDays = 0;
- C4 -
- C4 ----- //
- C4 ----- // Availability FIRST, then the UDC override (per Bill).
- C4 ----- // Avail was computed above from HDIPLT.
- C4 ----- // Avail = (IWOHQD + IWQ00) - (IWRESQ + IPCMT0);
- C4 ----- // If available -> honor the per item UDC override (or the
- C4 ----- // in stock default of 1 day). If short -> fall through to the
- C4 ----- // generic day count rules below.
- C4 ----- //
- C4 -
- C4 ----- If Avail >= 01Q0rd;
- C4 -
- C4 ----- // Available -> honor the per item UDC override
- C4 ----- Exec SQL
- C4 ----- Select Coalesce(Cast(UDCDESC1 As Decimal(2,0)), 0)
- C4 ----- Into :UDCDays :NullInd
- C4 ----- from ProITRG/UDCDETAIL
- C4 ----- Where UDCSYSTEMD = 'ORDERLINES' And UDCCODED = 'REQUIREDDAY'
- C4 ----- And UDCKEY = :01BItem;
- C4 -
- C4 ----- If UDCDays > 0; ----- // on the UDC table
- C4 ----- AddDays = UDCDays;
- C4 ----- Else; ----- // not on the table -> in stock default
- C4 ----- AddDays = 1;
- C4 ----- EndIf;
- C4 -
- C4 ----- Else;
- C4 -
- C4 ----- // Not available -> generic day count rules
- C4 ----- Select;
- C4 -
- C4 ----- When impels = 'DCCU' Or
- C4 ----- impels = 'DCSG';
- C4 -
- C4 ----- // KIT (Digital Cut) product class DCCU / DCSG -> 5 days
- C4 ----- AddDays = 5;
- C4 -
- C4 ----- When impels = 'KITS';
- C4 -
- C4 ----- // KIT (Digital / Screen Print) product class KITS -> 10 days
- C4 ----- AddDays = 10;
- C4 -
- C4 ----- When (%Subst(01BItem:1:1) = '9' And

```

```

- C4 ----- %Subst(01BItem:3:1) = ' ' ) Or
- C4 ----- %Subst(01BItem:1:3) = '3M0';
- C4 -
- C4 ----- // Stock item, NOT in stock -> 5 business days and notify CS
- C4 ----- AddDays = 5;
- C4 -
- C4 ----- Alert = 'XMLAvail';
- C4 ----- If xmlMode = 'T';
- C4 ----- AlertSubj = '*** TEST ***';
- C4 ----- EndIf;
- C4 ----- AlertSubj = %Trim(AlertSubj) + ' ' +
- C4 ----- 'Import Order Alert: STOCK ITEM AVAILABILITY';
- C4 ----- AlertMsg = 'Import Order Customer: ' + xmlRetailer + CrLf;
- C4 ----- AlertMsg = %Trim(AlertMsg) + 'HarrisData Order: '
- C4 ----- + %Char(LastOrder) + CrLf;
- C4 ----- AlertMsg = %Trim(AlertMsg) + 'Coupa Order: '
- C4 ----- + HWPO# + CrLf;
- C4 ----- AlertMsg = %Trim(AlertMsg) + 'Order Line: '
- C4 ----- + %Char(01OrL#) + CrLf;
- C4 ----- AlertMsg = %Trim(AlertMsg) + 'Item: '
- C4 ----- + 01BItem + CrLf;
- C4 ----- AlertMsg = %Trim(AlertMsg) + 'Issue: Availability'
- C4 ----- + CrLf;
- C4 ----- AlertMsg = %Trim(AlertMsg) + 'Details: ' +
- C4 ----- %Char(01QOrd) +
- C4 ----- ' Ordered Only ' + %Char(Avail) +
- C4 ----- ' are available.';
- C4 ----- Exsr Notify;
- C4 -
- C4 ----- Other;
- C4 -
- C4 ----- // Custom / non-stock item -> 10 days
- C4 ----- AddDays = 10;
- C4 -
- C4 ----- EndSl;
- C4 -
- C4 ----- EndIf;
- C4 -
- C4 -
- C4 ----- //*****//
- C4 ----- // Calculate the required date...//
- C4 ----- //*****//

```

```

1330 + C4 //=====
1331 + C4 // C1/C2 Hardcoded product-class branches (DCCU / DCSG / KITS)
1332 + C4 // and the 9x- / 3M0 stock-name test are removed. The
1333 + C4 // XMLAvail short-stock alert lived inside that block and
1334 + C4 // is removed with it.
1335 + C4 // C4 The required-date day count now comes entirely from the
1336 + C4 // UDC file, in this order:
1337 + C4 // 1. ORDERLINES / REQUIREDAY on the item
1338 + C4 // 2. ORDERLINES / PREFIXRQDD on the item prefix,
1339 + C4 // scoped to this customer's class, or the
1340 + C4 // all-customers key 'SCREEN GRAPHICS'
1341 + C4 // 3. ORDERLINES / PCREQUIRED on the product class
1342 + C4 // 4. HDIPLT.IPTLT, only when above 1 day

```

```

1343 + C4 // 5. default of 10
1344 + C4 // In stock always returns 1 business day.
1345 + C4 // CalcReqD skips weekends and HOLIDAY rows, so every
1346 + C4 // value set below is a business-day count.
1347 + C4 //=====
1348 + C4
1349 + C4 AddDays = 0;
1350 + C4
1351 + C4 If Avail >= 01Q0rd;
1352 + C4
1353 + C4 // In stock -> next business day. No lookup: an item with
1354 + C4 // exact or more available is always 1 day.
1355 + C4 AddDays = 1;
1356 + C4
1357 + C4 Else;
1358 + C4
1359 + C4 // ReqDays is zeroed first and each tier runs only while it is
1360 + C4 // still zero, so a no-row condition (SQLCODE 100) can never
1361 + C4 // carry a prior line's value forward.
1362 + C4 ReqDays = 0;
1363 + C4
1364 + C4 // 1. exact item
1365 + C4 Exec SQL
1366 + C4 Select Coalesce(Cast(UDCDESC1 As Decimal(3,0)),0)
1367 + C4 Into :ReqDays
1368 + C4 From ProITRG/UDCDETAIL
1369 + C4 Where UDCSYSTEMD = 'ORDERLINES'
1370 + C4 And UDCCODED = 'REQUIREDAY'
1371 + C4 And UDCKEY = :01BItem;
1372 + C4
1373 + C4 // 2. item prefix, scoped by customer class. UDCKEY holds
1374 + C4 // either a 2 character HDCCLS class code, which must match
1375 + C4 // this customer's CMCCLS, or the literal 'SCREEN GRAPHICS'
1376 + C4 // which applies to every customer. A customer specific row
1377 + C4 // outranks 'SCREEN GRAPHICS'; among equals the longest
1378 + C4 // prefix wins.
1379 + C4 If ReqDays = 0;
1380 + C4 Exec SQL
1381 + C4 Select Coalesce(Cast(UDCDESC1 As Decimal(3,0)),0)
1382 + C4 Into :ReqDays
1383 + C4 From ProITRG/UDCDETAIL
1384 + C4 Where UDCSYSTEMD = 'ORDERLINES'
1385 + C4 And UDCCODED = 'PREFIXRQDD'
1386 + C4 And :01BItem Like Concat(RTrim(UDCINDEXED),'%')
1387 + C4 And (UDCKEY = :CustClass Or
1388 + C4 UDCKEY = 'SCREEN GRAPHICS')
1389 + C4 Order By Case When UDCKEY = 'SCREEN GRAPHICS'
1390 + C4 Then 1 Else 0 End,
1391 + C4 Length(RTrim(UDCINDEXED)) Desc
1392 + C4 Fetch First 1 Row Only;
1393 + C4 EndIf;
1394 + C4
1395 + C4 // 3. product class, taken from the guarded HDIMST chain
1396 + C4 If ReqDays = 0;
1397 + C4 Exec SQL

```

```

1398 + C4      Select Coalesce(Cast(UDCDESC1 As Decimal(3,0)),0)
1399 + C4      Into :ReqDays
1400 + C4      From ProITRG/UDCDETAIL
1401 + C4      Where UDCSYSTEMD = 'ORDERLINES'
1402 + C4      And UDCCODED   = 'PCREQUIRED'
1403 + C4      And UDCKEY     = :WkClass;
1404 + C4      EndIf;
1405 + C4
1406 + C4      Select;
1407 + C4      When ReqDays > 0;
1408 + C4      AddDays = ReqDays;
1409 + C4
1410 + C4      When LeadDays > 1;
1411 + C4      // Plant lead time. IPTLT is DECIMAL(5,1), so round a partial
1412 + C4      // day up: a line cannot ship in part of a business day.
1413 + C4      // A value of 1 or less is a ship-from-stock figure and is
1414 + C4      // meaningless for a short line, so it falls through to the
1415 + C4      // default rather than promising next day on a short order.
1416 + C4      AddDays = %Int(LeadDays);
1417 + C4      If LeadDays > AddDays;
1418 + C4      AddDays += 1;
1419 + C4      EndIf;
1420 + C4
1421 + C4      Other;
1422 + C4      AddDays = 10;
1423 + C4
1424 + C4      EndSl;
1425 + C4
1426 + C4      EndIf;
1427      Exsr CalcReqD;
1428      //*****//
1429
1430      Chain (01Plt:1:01ITEM) HDMCMM;
1431      If %Found(HDMCMM);
1432      01UCC1 = CMUCC1;
1433      01UCC2 = CMUCC2;
1434      01UCC3 = CMUCC3;
1435      01UCC4 = CMUCC4;
1436      01UCC5 = CMUCC5;
1437      01COST = 01UCC1 + 01UCC2 + 01UCC3 + 01UCC4 + 01UCC5;
1438      EndIf;
1439
1440      01LIST = 01QORD * 01SlPr;
1441      OrdTotal += 01List;
1442      01QAFG = '1';
1443      01UDA1 = %Subst(giftMSG:1:15);
1444      01UDA1 = %XLate(Lo:Up:01UDA1);
1445      01UDA2 = %Subst(giftMSG:16:15);
1446      01UDA2 = %XLate(Lo:Up:01UDA2);
1447      01UDA3 = %Subst(giftMSG:31:15);
1448      01UDA3 = %XLate(Lo:Up:01UDA3);
1449      01UDA4 = %Subst(giftMSG:45);
1450      01UDA4 = %XLate(Lo:Up:01UDA4);
1451
1452

```

```

1453 Write OEDTWKR;
1454 + C13 ProcessedItems += 1; // C13
1455
1456 If ProcessMode = 'L';
1457     XDOBAT = HWPO#;
1458     XDSNDR = cXML.Header.From.Credential.Identity;
1459     // XDXORD = %Dec(XMLOrderID:11:0);
1460     XDXORD = 0;
1461     XDPO = HWPO#;
1462     XDHODR = LastOrder;
1463     XDOLIN = lineCounter;
1464     XDLNIID = %Char(lineCounter);
1465     XDMLin = %Char(cXML.Request.OrderRequest(orderCounter).
1466         ItemOut(lineCounter).lineNumber) ;
1467
1468     XDQty0 = %Dec(%XLATE('$*',' ':cXML.Request.
1469         OrderRequest(orderCounter).
1470         ItemOut(lineCounter).
1471         quantity) : 13 : 4);
1472
1473     XDUOM = 'EA';
1474     XDDESC = O1IMDS;
1475     XDMSKU = cXML.Request.OrderRequest(orderCounter).
1476         ItemOut(lineCounter).ItemID.SupplierPartID;
1477
1478     XDUCST = O1S1Pr;
1479     XDLSHP = O1SvSv;
1480     XDLSHP = %XLate(Lo:Up:XDLSHP);
1481     XDEShp = %char(LastDt1Date:*ymd0);
1482
1483 Write XMLOEDTR;
1484
1485 EndIf;
1486
1487 EndFor;
1488 + C13
1489 + C13 //=====
1490 + C13 // C13 Order arrived with no detail lines. Alert only. The order
1491 + C13 // IS still created: execution falls straight through to the
1492 + C13 // H1RqDt fallback, Write OEHDWKR, the tax call, the data queue
1493 + C13 // and the XMLOEHD write, exactly as it does today.
1494 + C13 // Do NOT add LeaveSR here.
1495 + C13 //=====
1496 + C13 If ProcessedItems = 0;
1497 + C13
1498 + C13     Alert = 'XMLNODTL';
1499 + C13     AlertSubj = *Blanks;
1500 + C13     If xmlMode = 'T';
1501 + C13         AlertSubj = '*** TEST ***';
1502 + C13     EndIf;
1503 + C13     AlertSubj = %Trim(AlertSubj) + ' ' +
1504 + C13         'Imported Order Alert: Order has no detail';
1505 + C13     AlertMsg = 'Customer: ' + %Trim(xmlRetailer) + CrLf;
1506 + C13     AlertMsg = %Trim(AlertMsg) + 'Imported File: ' +
1507 + C13         %Trim(ifsFile) + CrLf;

```

```

1508 + C13      AlertMsg = %Trim(AlertMsg) + 'Imported Order: ' +
1509 + C13      %Trim(HWPO#) + CrLf;
1510 + C13      AlertMsg = %Trim(AlertMsg) + 'Issue: No Items Ordered' + CrLf;
1511 + C13      Exsr Notify;
1512 + C13
1513 + C13      EndIf;
1514 + C13      //=====
1515
1516      // No line set a date -> fall back to the next business day
1517      If H1RqDt = 0;
1518          H1RqDt = %Dec(%Char(LastDtldDate:*CYMD0):7:0);
1519      EndIf;
1520
1521      // Notify Bill Busch if the order didn't have anything to process
1522      // in it. Skip the rest of the processing and do NOT write to
1523      // OEHDWK
1524      // If ProcessedItems = 0;
1525      // Notify Bill via email. End processing and move the file
1526      // to the archive
1527      // Alert = 'XMLNODTL';
1528      // If xmlMode = 'T';
1529      // AlertSubj = '*** TEST ***' ;
1530      // EndIf;
1531      // AlertSubj = %Trim(AlertSubj) + ' ' +
1532      // 'Imported Order Alert: Order has no detail';
1533      // AlertMsg = 'Customer: ' + xmlRetailer + CrLf;
1534      // AlertMsg = 'Imported File: ' + %Trim(ifsFile) + CrLf;
1535      // AlertMsg = %Trim(AlertMsg) + 'Imported Order: '
1536      // + HWPO# + CrLf;
1537      // AlertMsg = %Trim(AlertMsg) + 'Issue: No Items Ordered'
1538      // + CrLf;
1539      // Exsr Notify;
1540
1541      // Exit the subroutine and let the exitProgram routine archive
1542      // the file and exit to process the next order.
1543      // LeaveSR;
1544
1545      // EndIf;
1546
1547
1548      // Write the OEHDWK record with the order total...
1549
1550      H1OCTL = LastOrder;
1551      H1ORD# = LastOrder;
1552      H1MNCD = 'A';
1553      H1BLTO = HDPCust#;
1554
1555      Chain H1BLTO HDCUSTR;
1556 + C16
1557 + C16      // C16 - a failed chain must not leave the previous
1558 + C16      // customer's values on the order header.
1559 + C16      If NOT %Found(HDCUST);
1560 + C16          Clear HDCUSTR;
1561 + C16      EndIf;

```

1562

```

1563      H1Bank = 7;
1564      H1Loc# = CMLoc#;
1565
1566      H1SLSM = CMSLSM;
1567
1568      Clear HDTAXDR;
1569      Chain (STState: 0: 0: 0: 0: 0) HDTAXDR;
1570
1571      H1STTX = TDSTCD;
1572      If ShipToFnd <> 'Y';
1573          H1CTXC = CMCTXC;
1574      Else;
1575          H1CTXC = STTxCd;
1576      EndIf;
1577      H1CNTX = CMCNTC;
1578      H1CTTX = CMCTTC;
1579      H1LOC1 = CMLOC1;
1580      H1LOC2 = CMLOC2;
1581      H1LOC3 = CMLOC3;
1582      H1BDTE = %dec(%char(%date(%Subst(HWOrdDate:1:10):*iso-):*cymd0):7:0);
1583      H1ORTY = CMORTY;
1584
1585      H1ORRF = HWPO#;
1586      H1CTRM = CMCTRM;
1587      If ShipToFnd = 'Y';
1588          H1SVSV = CMSVSV;
1589      Else;
1590          H1SVSV = CMSV;
1591      EndIf;
1592
1593      // Pull the Ship Via Description
1594      Chain H1SVSV HDShPV;
1595      If %Found(HDShPV);
1596          H1SVDS = SVSVDS;
1597      Else;
1598          H1SVDS = *Blanks;
1599      EndIf;
1600
1601      H1BOAL = *Blanks;
1602      H1TOT = OrdTotal;
1603      H1FBCD = 'OR';
1604
1605      // Set Order Flags (Pulled directly from HOEOEM)
1606
1607      FGORTY = CMORTY;
1608
1609      /End-Free
1610      C*
1611      C      KFG      KLIST
1612      C      KFLD      KFLD      APID
1613      C      KFLD      FGORTY
1614      C*
1615      C      CLEAR      DFL
1616      C      KFG      SETLL      HDORFGR
1617      C      DOU      1=2

```

```

1618 C      KFG      READE      HDORFGR
1619 C              IF          %EOF(HDORFG)
1620 C              LEAVE
1621 C              ENDIF
1622 C              Z-ADD      FGFGFG      F
1623 C              MOVEL      FGFGOP      DFL(F)
1624 C              ENDDO
1625 C*
1626 C              MOVEA      DFL          H10EF
1627 C*
1628 /Free
1629
1630 Write OEHDWKR;
1631
1632
1633
1634 // Calculate taxes and add them to the order...
1635 xmlOCTL = LastOrder;
1636 xmlFRT = 0.00;
1637 xmlCHRG = 0.00;
1638 xmlUPDTOT = 'R';
1639 HOEUMC_W(xmlWHnd1 : xmlDBID : xmlOCTL : xmlFRT : xmlCHRG :
1640          xmlUPDTOT);
1641
1642
1643 // Notify customerservice@screen-graphics.com if this order had any
1644 // "CUSTOM ITEMS" on it. Specify all the normal detail information
1645 // but specify each "CUSTOM ITEM" detail line by number with its
1646 // full description, quantity, price, and extended price.
1647
1648 If CustomItem = 'Y';
1649 Exsr AlertCustom;
1650 EndIf;
1651
1652
1653 // Submit the order to the Data Queue...
1654 OQTYP = 'ORVW';
1655 QTURN = 0;
1656 QQORD# = LastOrder;
1657 QQOCTL = LastOrder;
1658 QQTSTP = %TimeStamp();
1659 QQTSTS = %SUBST(XMLRetailer:1:10);
1660 QQTSSW = 'XML850R_AR';
1661 QQPGM = 'HOEOEM';
1662 QSNDDTAQ(DQ#NAM:DQ#LIB:DQ#LEN:DTAQ);
1663
1664 If ProcessMode = 'L';
1665 // Now write the order to the XML Orders files...
1666 Clear XMLOEHDR;
1667
1668 XHFILE = XMLFILE;
1669 XHOBAT = XMLOrderID;
1670 XHPTNR = XMLRetailer;
1671 XHSNDR = 'Amerigas';
1672 XHTRNS = Transact;

```

```

1673      // XHXORD = %Dec(XMLOrderID:11:0);
1674      XHXORD = 0;
1675      XHPO   = HWPO#;
1676      XHHDOR = LastOrder;
1677      XHOLNT = HWLinCnt;
1678      XHPOTY = HWPOType;
1679      XHGFTM = HWGiftMSG;
1680      XHODTE = HWOrdDate;
1681      XSHSCD = HWShipCd;
1682      XHCNTL = HWControl;
1683      XHSLDV = HWSlsDiv;
1684      XHCORD = XMLOrderID;
1685      XHBCNT = HWBuyCntrct;
1686      XHMDPT = HWMerchDpt;
1687      XHRQSD = %EditC(H1RqDt:'X');
1688      // Capture Ship-To Address Only...
1689      XSHPPP = HWShpToID;
1690      XHPPNM1 = STName1;
1691      XHPP1A1 = STAdr1;
1692      XHPP1A2 = STAdr2;
1693      XHCTY1 = STCity;
1694      XHST1  = STState;
1695      XHCNT1 = STCountry;
1696      XHZIP1 = STZIP;
1697      XHEML1 = STEmail;
1698      XHDPH1 = STPhone;
1699      XHBLPP = HWBillToID;
1700      XHCUPP = HWCustID;
1701      XHIVPP = HWInvToID;
1702
1703      Write XMLOEHDR;
1704
1705      EndIf;
1706
1707      EndSr;
1708      //-----*
1709
1710
1711
1712      //-----*
1713      // WriteCmt will handle writing detail line item comments to an *
1714      // order. *
1715      //-----*
1716
1717      BegSr WriteCmt;
1718
1719      For x = 1 to 5;
1720          O4CSEQ = x;
1721          Select;
1722              When x = 1;
1723                  O4CMNT = %Subst(SaveDesc : 1 : 60);
1724              When x = 2;
1725                  O4CMNT = %Subst(SaveDesc : 61 : 60);
1726              When x = 3;
1727                  O4CMNT = %Subst(SaveDesc : 121 : 60);

```

```

1728     When x = 4;
1729         04CMNT = %Subst(SaveDesc : 181 : 60);
1730     When x = 5;
1731         04CMNT = %Subst(SaveDesc : 241 : 60);
1732     EndSl;
1733
1734     If 04CMNT <> *Blanks;
1735         04OCTL = LastOrder;
1736         04ORL# = lineCounter;
1737         04BLN# = 0;
1738         04DOCT = 'ACK';
1739         04CSR = *Blanks;
1740
1741         Chain (04OCTL:04ORL#:04BLN#:04DOCT:04CSEQ) OECMWKR;
1742         If NOT %Found(OECMWK);
1743             Write OECMWKR;
1744         EndIf;
1745     EndIf;
1746 EndFor;
1747
1748 EndSr;
1749 //-----*
1750
1751
1752
1753
1754 //-----*
1755 /* AlertCustom notifies customerservice@screen-grpahics.com of *
1756 /* the orders with 'CUSTOM ITEM's on them and the details about *
1757 /* those lines. *
1758 //-----*
1759 BegSr AlertCustom;
1760
1761 Select;
1762     When ProcessMode = 'T';
1763         AlertSubj = 'Testing Amerigas archived order ' +
1764             'file ' + %Trim(xmlFile) + ' with ' +
1765             'a custom item.';
1766     When xmlMode = 'T';
1767         AlertSubj = 'Amerigas sent a test order in file ' +
1768             %Trim(xmlFile) + ' with a custom item.';
1769     Other;
1770         AlertSubj = 'Amerigas has at least one custom ' +
1771             'item in order ' + %Trim(HWPO#) +
1772             ' in file ' + %Trim(xmlFile);
1773     EndSl;
1774     AlertMsg = %Trim(AlertMsg) + 'HarrisData Order: '
1775         + %Char(LastOrder) + CrLf;
1776
1777
1778 // Now pull the order from OEDTWK, each 'CUSTOM ITEM' line,
1779 // and report all the details about those lines for customer service.
1780 Exec SQL
1781     Declare Custom Cursor for
1782         Select 01ORL#, 01RQDT, 01IMDS, 04CMNT, 01QORD, 01SLPR

```

```

1783         from OEDTWK Left Outer Join OECMWK on
1784             O1OCTL = O4OCTL And O1ORL# = O4ORL# And
1785             O4DOCT = 'INT' And O4CSEQ = 1
1786         Where O1ITEM = 'CUSTOM ITEM' and
1787             O1OCTL = :LastOrder ;
1788
1789     Exec SQL
1790         Open Custom;
1791
1792     Exec SQL
1793         Fetch Next from Custom Into :CustOrdLn, :CustRqDt,
1794             :CustDesc, :CustCmt,
1795             :CustQOrd, :CustSlPr;
1796
1797     pass = 0;
1798
1799     DoW SQLCod = 0;
1800
1801     pass += 1;
1802
1803     If pass > 1;
1804         AlertMsg = %Trim(AlertMsg) + CrLf;
1805     EndIf;
1806
1807     AlertMsg = %Trim(AlertMsg) + 'Order Line: '
1808         + %Char(CustOrdLn) + CrLf;
1809     AlertMsg = %Trim(AlertMsg) + 'Required: '
1810         + %Char(CustRqDt) + CrLf;
1811     AlertMsg = %Trim(AlertMsg) + 'Description: '
1812         + %Trim(CustDesc) + %Trim(CustCmt) +
1813         CrLf;
1814     AlertMsg = %Trim(AlertMsg) + 'Qty: ' +
1815         %Char(CustQOrd) + CrLf;
1816     AlertMsg = %Trim(AlertMsg) + 'Price: ' +
1817         C14 %Char(CustSlPr) + CrLf +
1818         + C14 %Char(CustSlPr) + CrLf ;
1819     AlertMsg = %Trim(AlertMsg) + 'Extended Price: ' +
1820         %Char(CustQOrd * CustSlPr);
1821
1822     Exec SQL
1823         Fetch Next from Custom Into :CustOrdLn,
1824             :CustRqDt, :CustDesc, :CustCmt,
1825             :CustQOrd, :CustSlPr;
1826
1827     EndDo;
1828
1829     Exec SQL
1830         Close Custom;
1831
1832     Exsr Notify;
1833
1834     EndSr;
1835     //-----*
1836

```

```

1837 //-----*
1838 /* exitProgram archives the order processed and terminates the *
1839 /* program upon completion *
1840 //-----*
1841 BegSr exitProgram;
1842
1843 // Archive the order file...
1844
1845 Cmd = 'CPY OBJ(' + '''' + %Trim(filename) + '''' + ') ' +
1846 'TODIR(' + '''' + '/orders/archive inbound/' +
1847 %Trim(XMLRetailer) + '/orders' + '''' + ') ' +
1848 'REPLACE(*YES) OWNER(*NEW)';
1849 CmdLen = %Len(%Trim(Cmd));
1850 ExecCmd (Cmd:CmdLen);
1851
1852 Cmd = 'RMVLNK OBJLNK(' + '''' + %Trim(filename) + '''' + ')';
1853 CmdLen = %Len(%Trim(Cmd));
1854 ExecCmd (Cmd:CmdLen);
1855
1856 EndSr;
1857 //-----*
1858
1859 // -----//
1860 // NxthDOrd subroutine will pull the last order number Harris //
1861 // used and update it for the new cXML order. It will also //
1862 // write to the XML cross reference file of orders. //
1863 // -----//
1864 BegSR NxthDOrd;
1865
1866 Exec SQL
1867     Select CELORD into :LastOrder from OECtrl;
1868
1869 LastOrder += 1;
1870
1871 Exec SQL
1872     Update OECTRL set CELORD = :LastOrder;
1873
1874 CurDate = %Date();
1875 CurTime = %Time();
1876
1877 Exec SQL
1878     Insert into XMLORDS (XCCOORD, XHDSORD, XPROCISO, XPROCHMS)
1879     Values (:XMLOrderID, :LastOrder, :CurDate, :CurTime);
1880
1881 // Calculate the required date for the order
1882 TempDate = %Date() + %Days(1);
1883
1884 // *Mon = 0 ... Sun = 6
1885 Day_of_Week = %rem(%diff(%date() +
1886 %Days(1):d'0001-01-01':*days):7);
1887
1888 If Day_of_Week = 5; //Saturday

```

```

1892         TempDate = %Date() + %Days(3);
1893     EndIf;
1894     If Day_of_Week = 6;    // Sunday
1895         TempDate = %Date() + %Days(2);
1896     EndIf;
1897
1898     LastDtldate = TempDate;
1899     // Seed header date to zero; CalcReqD sets it to the earliest
1900     // line required date. Empty orders fall back after the loop.
1901     H1RqDt = 0;
1902
1903 EndSR;
1904 // -----//
1905
1906
1907
1908 // -----//
1909 // CalcReqD subroutine will calculate the required date for //
1910 // individual detail line item based on Bill's criteria of //
1911 // the number of days to add based on the item ordered. It //
1912 // will also take into account any holidays and weekends and //
1913 // not include those in the data calculations. //
1914 // -----//
1915 BegSR CalcReqD;
1916
1917     TodayISO = %Date();
1918     TempDate = TodayISO;
1919
1920     // Add AddDays *business* days: step one calendar day at a
1921     // time, skipping Saturdays, Sundays, and holidays until
1922     // AddDays working days have been counted. The order date
1923     // itself is not counted (Tue + 1 business day = Wed).
1924     DoW AddDays > 0;
1925
1926     TempDate = TempDate + %Days(1);
1927
1928     // Mon = 0 ... Sun = 6
1929     Day_Of_Week = %rem(%diff(TempDate:
1930         d'0001-01-01':*days):7);
1931     If Day_Of_Week = 5 Or Day_Of_Week = 6;    // Sat / Sun
1932         Iter;
1933     EndIf;
1934
1935     // Company holiday?
1936     Exec SQL
1937         Select Count(*) Into :DaysOff
1938         From PROITRG/UDCDetail
1939         Where Date(UDCIndexed) = :TempDate
1940         And UDCSystemD = 'HOLIDAY';
1941     If DaysOff > 0;
1942         Iter;
1943     EndIf;
1944
1945     AddDays -= 1;    // a real working day
1946

```

```

1947         EndDo;
1948
1949         // TempDate builds the actual O1RqDt...
1950         O1RqDt = %Dec(%Char(TempDate:*CYMD0):7:0);
1951
1952         // Header required date = earliest of the line required dates
1953         If H1RqDt = 0 Or O1RqDt < H1RqDt;
1954             H1RqDt = O1RqDt;
1955         EndIf;
1956
1957     EndSR;
1958
1959
1960     // -----//
1961     // ClearOrd subroutine will clear all the processing for this //
1962     // PO# since the order is rejected due to a bad item. //
1963     // -----//
1964     BegSR ClearOrd;
1965
1966     Exec SQL
1967         Delete From OEHDWK Where H1ORRF = :HWPO#;
1968
1969     Exec SQL
1970         Delete From OEDTWK Where O1OCTL = :LastOrder;
1971
1972     // Notify users of the cancellation....
1973     Exsr Notify;
1974
1975     EndSR;
1976     // -----//
1977
1978
1979
1980     // -----//
1981     // Notify subroutine will send notification of rejection for //
1982     // the order to the appropriate individuals. //
1983     // -----//
1984     BegSR Notify;
1985
1986     Exec SQL
1987     Declare Emails Cursor for
1988         Select UDCDESC1 from PROITRG/UDCDetail
1989         Where UDCSYSTEMD = 'EMAIL' and UDCCODED = :Alert;
1990
1991     Exec SQL
1992         Open Emails;
1993
1994     Exec SQL
1995         Fetch Next from Emails into :Distribution;
1996
1997     DoW SQLCod = 0;
1998
1999     // Actual email routine
2000     from = 'hdfiles@screen-graphics.com';
2001     to   = Distribution;

```

```

2002      replyTo = *Blanks;
2003      subject = AlertSubj;
2004      text = AlertMsg;
2005
2006      Monitor;
2007          email = RPGMail_new();
2008          RPGMail_addAddress(email:'FROM':from: ' ');
2009          RPGMail_addAddress(email:'TO':to: ' ');
2010          RPGMail_setSubject(email:subject);
2011          RPGMail_setBodyText(email: text: 'text/plain');
2012          RPGMail_setSMTPHost(email: 'MAILGATE');
2013          RPGMail_setSMTPPort(email: '25');
2014          RPGMail_send(email);
2015      On-error;
2016          error = Error_catch();
2017          // Don't swallow the failure - log it to the job log.
2018          LogTxt = 'XML850R_AR: e-mail send failed for ' + %Trim(to) +
2019                  ' (alert ' + %Trim(Alert) + '). ' +
2020                  %Trim(error.text);
2021          SndPgmMsg( *blanks : *blanks : LogTxt
2022                   : %Len(%Trim(LogTxt)) : '*DIAG' : '*PGMBDY' : 1
2023                   : MsgKey : ApiErr );
2024      EndMon;
2025
2026      Exec SQL
2027          Fetch Next from Emails into :Distribution;
2028
2029      EndDo;
2030
2031      Exec SQL
2032          Close Emails;
2033
2034      +   C20
2035      +   C20      // C20 - reset so the next alert cannot inherit this
2036      +   C20      //           one's subject or body text.
2037      +   C20      Clear Alert;
2038      +   C20      Clear AlertSubj;
2039      +   C20      Clear AlertMsg;
2040      EndSR;
2041      // -----//

```